THE UNIVERSITY OF WINNIPEG

APPLIED COMPUTER SCIENCE

Course Number - ACS-3931-001 Course Name – Principles of Operating Systems

Assignment #2 X11 forwarding, 'c' programming for simple semaphores, python access via X11 forwarding, and book problems

Today's Date: February 3rd, 2026 – midnight (Nexus clock)
Due Date: March 3rd, 2026 – 11:59:00 pm (Nexus clock)

IMPORTANT NOTES:

Submit all assignment results as described below in one Zip file via Nexus named 'assignment2_<studentID>.zip'.

Create only one pdf for all screenshots and text report(s) identifying clearly the Part number the screenshot / text that they belong to. If more than one pdf file is in the zip file, then 25 points will be deducted.

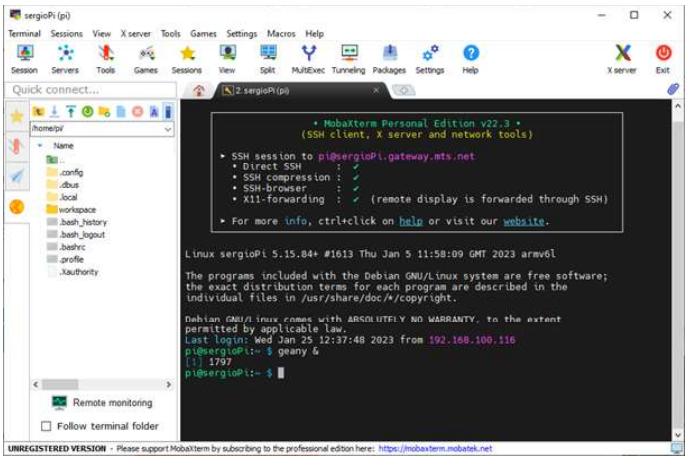
We are checking that mars and/or Pi screenshots have your username (in mars login) or student id number (in pi hostname) respectively to uniquely identify your ssh session when you run your programs, otherwise it will not be accepted as deliverable.

The assignment is worth 100 points + 20% optional if assignment is done on Raspberry Pi, e.g. if one gets 75 then 20% will be added to mark 75 or 15 pts in this case for a final mark of 90 pts total. One can get more than 100 pts for an assignment.

Part I (10 points)
In this Part I, you will practice X11 forwarding on mars / Pi from a MobaXTerm ssh client session to have a better development environment with minimum use of resources.

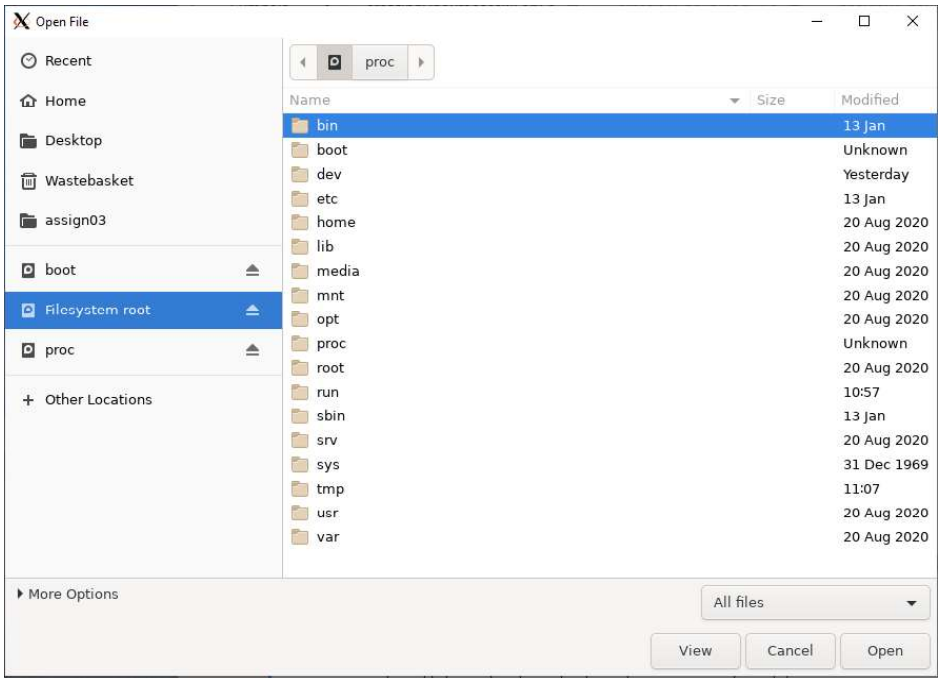
Submit via Nexus the following for Part I of the assignment:

- Screenshot for the ssh session showing geany running in the background on mars / Pi. i.e., a screenshot like



- Screenshot for the 'Open File' dialog as follows:
  a) Once geany is running on your desktop, go to 'File' -> 'Open…'
  b) In the geany 'Open File' window (depending what you're running):
  - On mars, click on the 'Other locations' -> 'Computer'
  - On the pi click on the 'Filesystem root' name (NOTE: DO NOT CLICK ON THE 'Unmount' ICON)

  and take the screenshot to report like the following



c) after you have the screenshot for the report, click 'Cancel' to go back to geany main window.

Part II (35 points)
In Parts II and III, you will practice basic threads with semaphores by writing two small c programs in geany.

The c program of Part II must do the following:

a) You will write a simple c program named 'assign02_noSem.c'. This program must

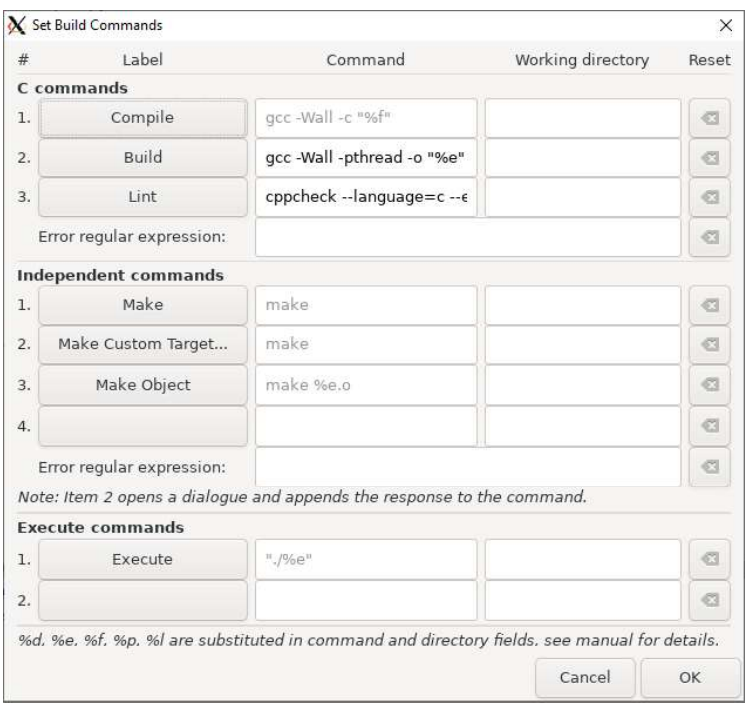
- Create a thread function named 'thread_hi_nosem' that implements the code.

```
for (int i=0; i<50; i++) {
      printf(".");
}
printf(" hi there\n");
```

- Create a thread function named 'thread_<myname>_nosem' that implements the code.

```
for (int i=0; i<50; i++) {
      printf("x");
}
printf(" <myname> \n");
```

- Include pthread.h and compile with the -pthread flag. In geany, setup the flag by selecting 'Build' -> 'Set Build Commands', once you have the 'Set Build Commands' dialog, enter the flag next to the 'Build' button as shown below.

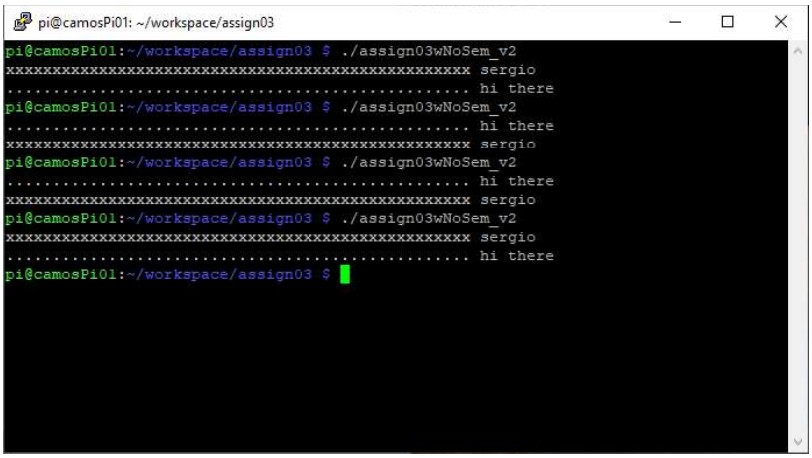


- The main program should create and run two threads. One for each thread function above with pthread_create(). You can learn more info on pthread.h including pthread_create() here https://pubs.opengroup.org/onlinepubs/7908799/xsh/pthread.h.html

- Once the threads are created, the main program should wait until the threads are finished with pthread_join().

- Compile, link and run your 'assign02_noSem.c' program.

- Run several times your program and analyze the results. Below a screenshot of a typical run



Submit via Nexus the following for Part II of the assignment:

- Source code for c program named 'assign02_noSem.c'. Important to add your name and student number as a comment at the beginning of the *.c file
- Briefly explain in a small paragraph (2 or 3 lines) why sometimes 'hi there' line prints after '<your_name>' line and sometimes not. Put the paragraph in the assignment pdf identifying that the answer is for Part II
  NOTE: if one can't see the interleaving issue, try increasing the counter (highlighted in yellow) to a big number, e.g., 100, 300, 500, 1000, etc. to make sure processor is given to another thread.
- Screenshot of your program running several times as shown in the previous shot and add it to the assignment pdf
- Put all *.c program files in the assignment2.zip file. Add this part's pdf to the zip file until all answers are put together in one final pdf.

Part III (35 points)

In Part III, you will fix the problem encountered in Part II to always print properly 'hi there' line before <your_name> line. You will use a semaphore to fix this problem.

The c program in Part III must do the following:

a) You will create another simple c program named 'assign02_Sem.c' that builds upon the 'assign02_noSem.c' program done in Part II. This program must

- Include semaphor.h You can learn more info on semaphore.h including its library functions here https://pubs.opengroup.org/onlinepubs/7908799/xsh/semaphore.h.html

- Rename the thread function named 'thread_hi_nosem' to 'thread_hi_sem' and the thread function named 'thread_<myname>_nosem' to 'thread_<myname>_sem'

- Add a global semaphore named 'mutex'. Semaphores are created as a variable of type sem_t

- You will use semaphore 'mutex' to make sure the 'hi there' line is always printed before '<your_name>' line. You must insert semaphore functions in the proper place in the two thread functions. Furthermore, you must initialize the semaphore with a proper value and destroy it in the main function. The following four semaphore functions can be useful to make the semaphores to work properly to achieve the required thread synchronization. You can see more info of these semaphore library functions at https://pubs.opengroup.org/onlinepubs/7908799/xsh/semaphore.h.html

- Semaphores are initialized with sem_init()

```
int sem_init(sem_t *sem, int pshared, unsigned int value);
                    ^            ^                    ^
                    |            |                    |
                    |            |                    |
   pointer to semaphore   0 for threads        initial value

 usage example:
        sem_t foo;                ← same global declaration in the global area for all examples

        sem_init(&foo,0,0);
```

- A semaphore is blocked with sem_wait()

```
int sem_wait(sem_t *sem);

usage example:
        sem_wait(&foo);
```

- A semaphore is incremented with sem_post()

```
int sem_post(sem_t *sem);

usage example:
        sem_post(&foo);
```

- A semaphore is destroyed with sem_destroy()

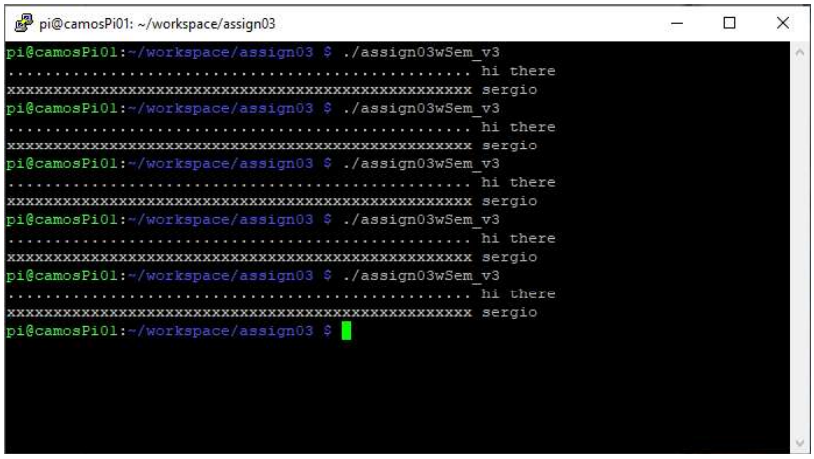
```
int sem_destroy(sem_t *sem);
```

usage example:

sem_destroy(&foo)

- Compile, link and run your ‘assign02_Sem.c’ program.

- Run several times your program and analyze the results. Below a screenshot of a typical run



No matter how many times you run it. You always get the proper synchronization thus the I/O access to the screen is in the proper order, and the text is always printed correctly.

Submit via Nexus the following for Part III of the assignment:

- Source code for c program named ‘assign02_Sem.c’. Important to add your name and student number as a comment at the beginning of the *.c file
- Briefly explain in a small paragraph (2 o 3 lines) what exactly the semaphore allows you to achieve with its use. Put the paragraph in the assignment pdf identifying that the answer is for Part III
- Screenshot of your program running several times as shown in the previous shot and add it to the assignment pdf
- Put all *.c files as Part III in the assignment2 zip file. Add this part’s pdf to the zip file until all answers are put together in one final pdf.

Part IV (20 points)

In this Part IV, you will connect to Python version 3.x on mars / Pi via X11 forwarding. With Python and c environments configured on mars / Pi via X11 forwarding, you have a very powerful development environment with minimum use of resources. With Python scripting capability (i.e., no need to compile) you can enter Python instructions on mars / Pi, which are directly interpreted. This becomes useful when you interface the computer with external hardware.

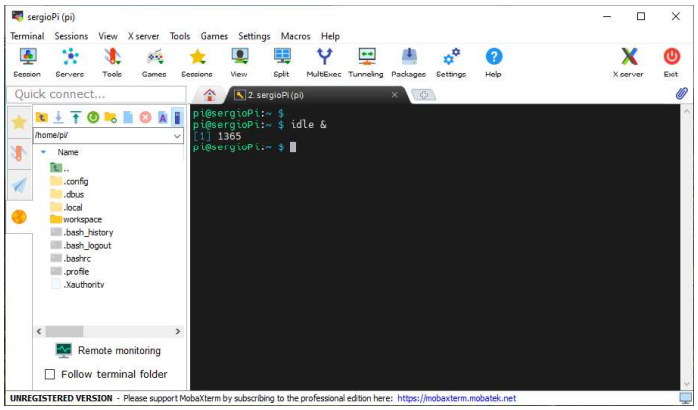
To do this Part IV, follow the following steps:

- Run MobaXterm SSH session to mars / pi
- Run idle client to access python interpreter by entering
    - idle &

NOTE: If working with the Pi, you may need to install additional software. Just follow the instructions shown on the screen to install missing software modules.

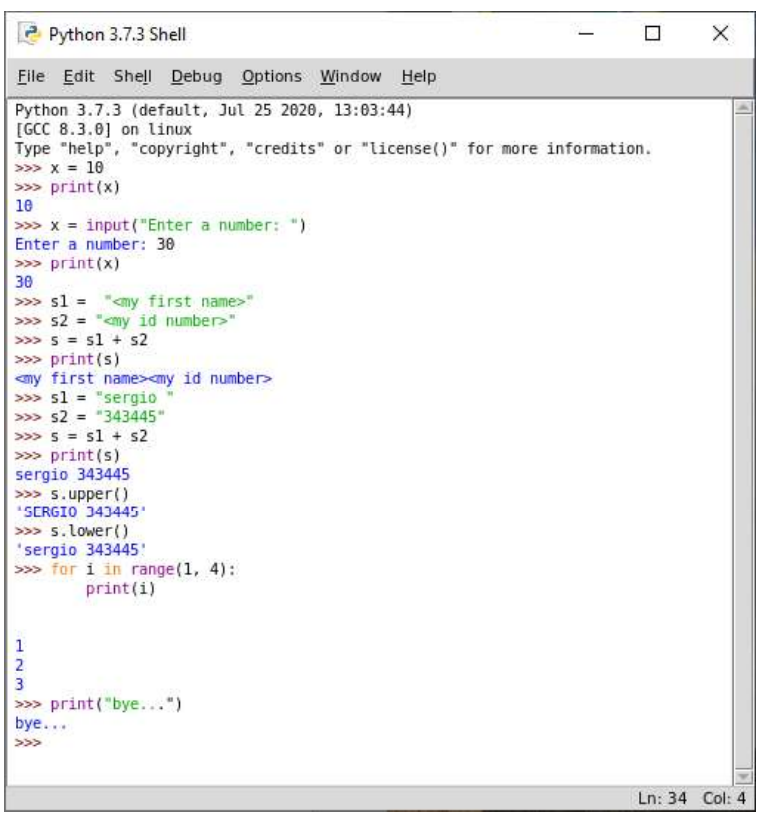
Submit via Nexus the following for Part IV of the assignment:

- Screenshot for the ssh session showing idle running in the background on mars / Pi. i.e., a screenshot like



Note that we are checking that the Pi hostname has your name or student id number to uniquely identify your Pi, otherwise it will not be accepted as deliverable.

- IDLE screenshot running all the sample Python instructions shown below. Also, you must use your name and student id number for s1 and s2 strings listed below instead.



Part V (0 points) – Nothing to report for this Part VI. This is additional material for midterm / final exams study.

In this Part VI, you will do some problems from your Stallings OS textbook 9th edition. NOTE: If you're using the International book edition, make sure you read & solve the respective problems of the North American book edition. If not, your problems will not correspond to the one listed below.

a) Read textbook as follows.

- Chapter 4 all sections with exception of the Windows, Solaris, Android and Mac OS sections

b) Work on the following book problems for exam practice.

1. Problem 4.1 Page 190
2. Problem 4.2 Page 191
3. Problem 4.5 Page 191
4. Problem 4.12 Page 194
5. Problem 4.7 Page 191-192